

# Trabajo Práctico: Test de Carga, Estrés y Optimización de Microservicio

## Contexto y Objetivos del Trabajo Práctico

El objetivo de este trabajo práctico es diseñar, implementar y optimizar el **microservicio de extracción de texto y conversión de PDF a Markdown**, evaluando su resiliencia, latencia y rendimiento bajo condiciones extremas de concurrencia mediante pruebas de carga y estrés con **Grafana k6** y **Vegeta**.

El Grupo de Desarrollo (Alumnos agrupados según el proyecto presentado) debe aplicar patrones de microservicios, **Twelve-Factor App**, control de concurrencia, gestión de recursos en contenedores (CPU/memoria) y técnicas de mitigación de saturación (backpressure, colas, paralelismo eficiente).

## Requerimientos del Microservicio

- **Endpoint obligatorio:** POST /extract
- **Entrada (Request):** Archivo binario PDF (mediante multipart/form-data o payload binario directo en el body).
- **Salida (Response):** application/json con código HTTP 200 OK cuyo contenido estructurado incluya el texto extraído en formato:

```
{  "content": "",  "page_count": 292}
```
- **Libertad de Stack:** Se permite el uso de cualquier lenguaje de programación, framework y biblioteca de procesamiento de PDF (Go, Rust, Java/Kotlin, C++, Python, Node.js, etc.).

## Condiciones del Entorno de Ejecución (Para igualdad de condiciones)

Para que los resultados de las pruebas sean comparables y justos respecto a la aplicación del profesor:

1. **Contenerización:** La aplicación debe empaquetarse en una imagen Docker y desplegarse mediante docker compose.
2. **Escalabilidad horizontal controlada:**  
El sistema puede escalar horizontalmente **hasta un máximo de 5 réplicas/instancias** (p. ej. deploy.replicas: 5 o balanceador inverso tipo Traefik/Caddy).

3. **Límites de hardware (Resource Limits):**

Cada contenedor/instancia debe contar con un límite explícito de recursos en docker-compose.yml (por ejemplo, máx. 1.0 CPU y 512 MB a 1 GB de RAM por réplica) para evitar disparidades.

4. **Mismo set de datos de prueba:**

Las pruebas deben ejecutarse utilizando la carpeta oficial de PDFs de prueba (tests/stress/pdfs), que contiene archivos de tamaño y densidad variable (desde livianos hasta PDFs con gráficos/capas de ~9 MB).

## Benchmark de Referencia del Profesor

Para obtener el puntaje extra, los grupos deberán superar el comportamiento del microservicio de referencia propuesto por la cátedra.

### A. Prueba Spike con Grafana k6 (Concurrencia Pico - Modelo Cerrado)

- **Perfil de carga:** Subida súbita a 100 VUs en 10s, 20s sostenidos a 100 VUs, rampa descendente de 10s a 0 VUs.
- **Métricas del profesor:**
  - **Peticiones procesadas:** 1.037 peticiones en 40 segundos.
  - **Throughput sostenido:** 25.35 req/s.
  - **Tasa de error:** 0.00% (100% respuestas HTTP 200 OK).
  - **Latencia Mediana (\$p\_{50}\$):** 1.88 s.
  - **Latencia P90 / P95:** 7.83 s / 8.80 s.
  - **Latencia máxima:** 13.94 s.

### B. Prueba de Carga Fija con Vegeta (Throughput Constante - Modelo Abierto)

- **Perfil de carga:** 50 req/s continuas durante 30 segundos (1.500 solicitudes generadas, rotando los 4 PDFs de prueba).
- **Timeout del cliente:** 30 segundos.
- **Métricas del profesor:**
  - **Throughput efectivo completado:** 16.65 req/s.
  - **Peticiones exitosas:** 998 / 1.500 (66.53%).
  - **Timeouts de cliente (código 0):** 501 peticiones (33.40%) debido a saturación de la cola de CPU al sobrepasar las ~25 req/s sostenibles.
  - **Latencia Mediana (P50):** 14.89 s.

**Nota técnica para los alumnos:** Observar la diferencia entre el comportamiento de un **modelo cerrado (k6)** donde las peticiones se serializan por usuario virtual, y un **modelo abierto (Vegeta)** donde la tasa de inyección es constante. El desafío es cómo el microservicio maneja la congestión sin generar tiempos de respuesta superiores al timeout.

## 5. Pautas y Desafíos de Optimización Sugeridos

Para superar la marca del profesor, se recomienda a los alumnos investigar sobre:

1. **Eficiencia en la extracción:** Librerías de bajo consumo de CPU y memoria que lean el stream sin volcar todo el archivo a disco ni duplicar buffers en memoria.
2. **Backpressure y Control de Concurrency:** Evitar que se acumulen 500 peticiones en cola que van a expirar; manejar límites de concurrencia y devolver 429 Too Many Requests o 503 Service Unavailable de forma controlada si la cola excede el tiempo útil de vida.
3. **Estrategia de réplicas y Balanceo:** Configurar las 5 réplicas detrás de un reverse proxy con balanceo.
4. **Pool de workers de procesamiento:** Separar el runtime HTTP de la ejecución del algoritmo de extracción de texto.

## 6. Criterios de Evaluación y Sistema de Puntos

| Concepto                               | Detalle                                                                                                                                                     | Puntaje                                  |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| <b>Puntaje Base del TP</b>             | Aprobación técnica, ejecución en Docker (hasta 5 réplicas), cumplimiento funcional ( <b>/extract</b> devolviendo Markdown) y scripts de prueba funcionales. | <b>Nota regular del TP</b>               |
| <b>Superar al Profesor</b>             | Lograr un throughput mayor a en k6, mejorar la tasa de éxito y latencias en Vegeta bajo las mismas restricciones de hardware.                               | <b>+1.5 puntos en el 2° Parcial</b>      |
| <b>Primer Puesto (Mejor resultado)</b> | El grupo que obtenga el mejor balance de throughput, tasa de fallos y percentiles de latencia entre todas las entregas.                                     | <b>+1.0 punto extra en el 2° Parcial</b> |

### Evaluación Cualitativa:

1. **Calidad de Código y Buenas Prácticas:**
  - Cumplimiento de principios de **Twelve-Factor App** (configuración vía variables de entorno, port binding, procesos sin estado, logs a stdout).
  - Estructuración del código, modularidad y legibilidad.
2. **Documentación e Informe Técnico:**

- Explicación de la arquitectura de la solución y decisiones de diseño.
- Comparativa de métricas antes vs. después de optimizar.
- Explicación detallada de **cuál fue el proceso de investigación** que los llevó a su configuración final (imprescindible para adjudicar el puntaje ganador).

## Entregables Requeridos

1. **Repositorio de código:** Con Dockerfile y docker-compose.yml reproducible con un único comando (docker compose up --build).
2. **Scripts de prueba:**
  - Script de k6 adaptado para reproducir el test de Spike.
  - Script de Vegeta para la prueba de carga fija.
3. **Informe en Markdown / PDF:**
  - Justificación de la arquitectura elegida y análisis del cuello de botella identificado.